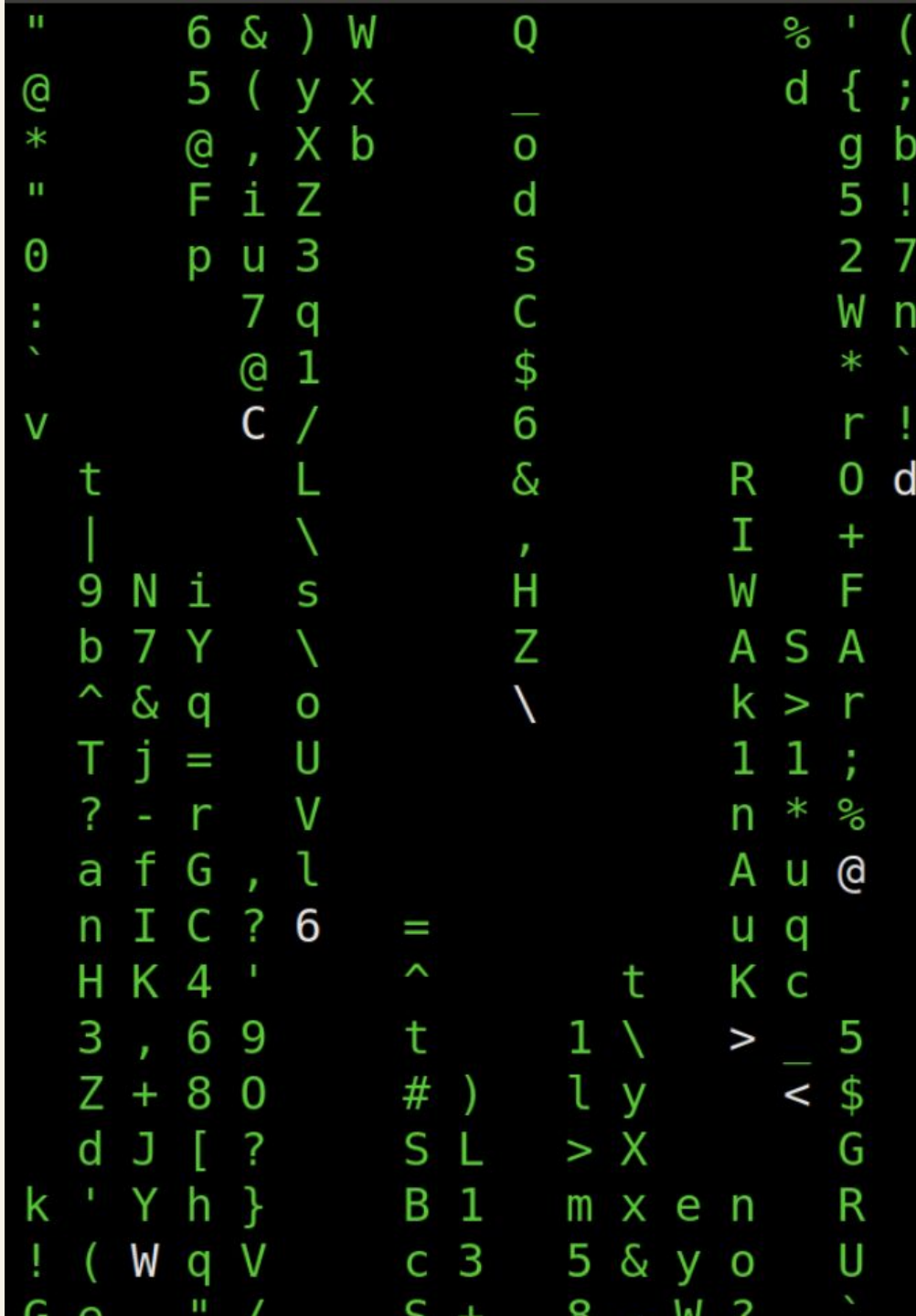
INTRO TO COMPUTER SCIENCE & CODING LOGIC TUTORIAL

Intro to the Terminal

(A gentle first look)

Sarah Al Mamoun



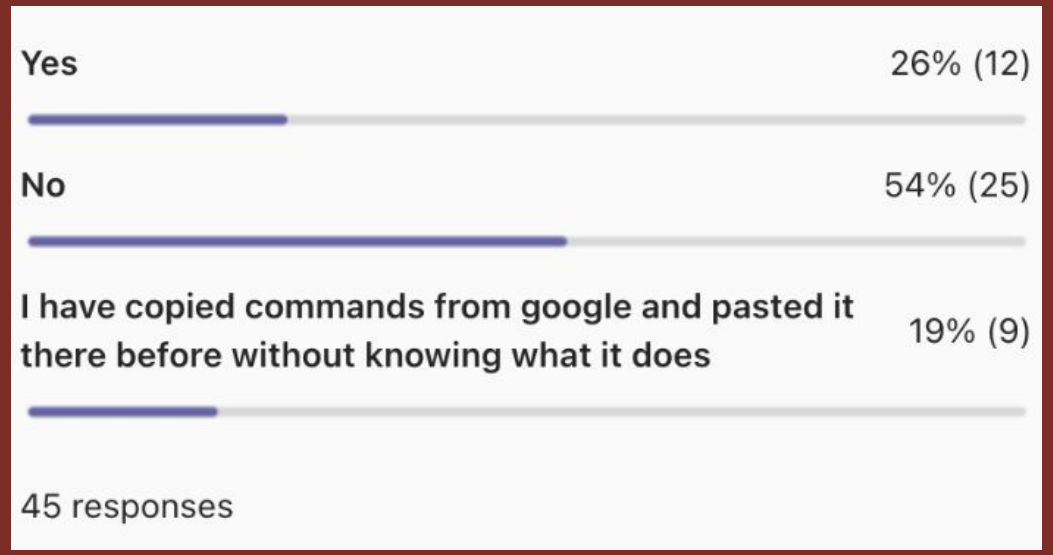
What have you used the terminal for before?

A
Never opened it

B
Pasted a command from a tutorial

C
Installed or ran something

D
I use it all the time



Stats from the first class

Today's plan

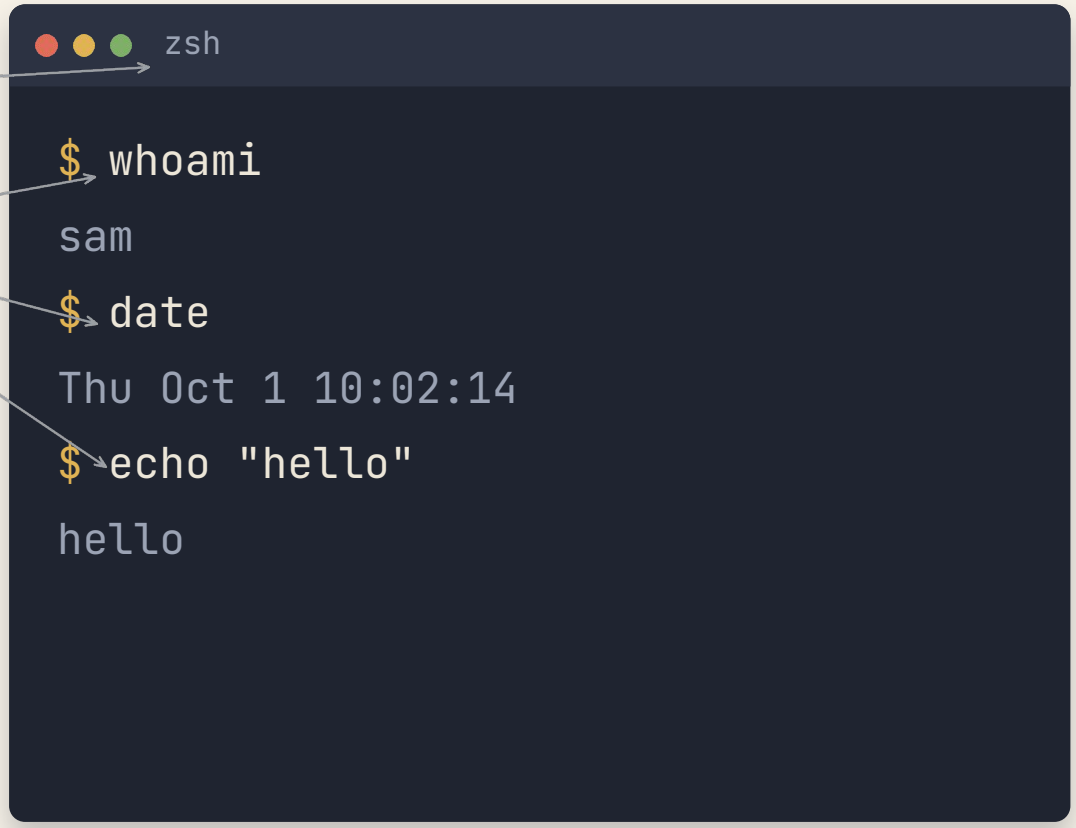
1. What a terminal is, and where to find one
2. The basic moves
3. Our first project
4. AI tools & git

A terminal is a conversation in text

Terminal the window you type in

Shell the program inside that understands you (mine is zsh)

Command line the line after the $ where you type



You type a command. The computer types back. No icons, no GUI, no clicking.

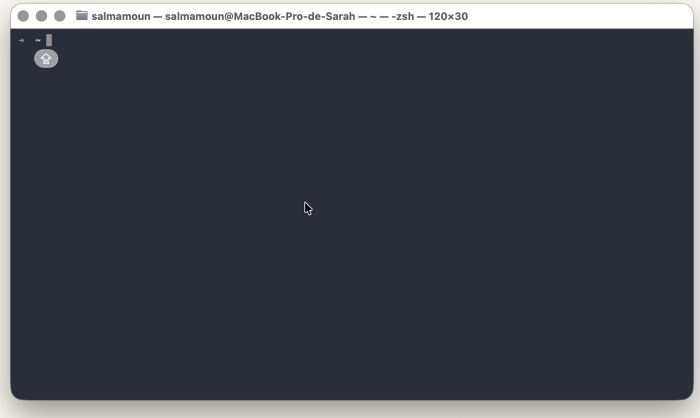
1. **Type** a command
2. **Press Enter** to run it
3. **Read** what comes back

Where do terminals live?

On its own

Mac: Terminal, iTerm2, Ghostty.

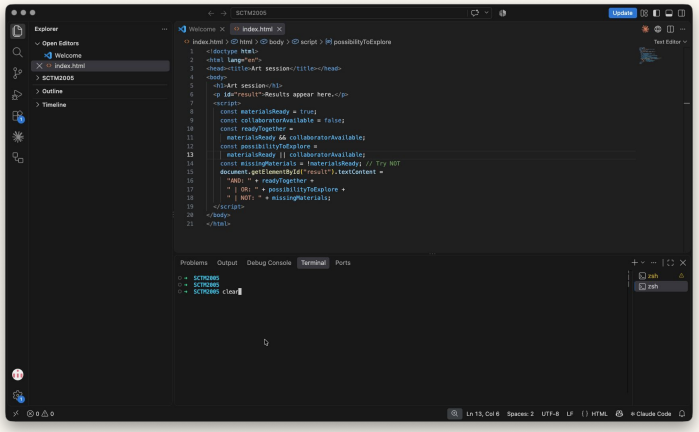
Windows: Windows Terminal.



Windows: check git bash

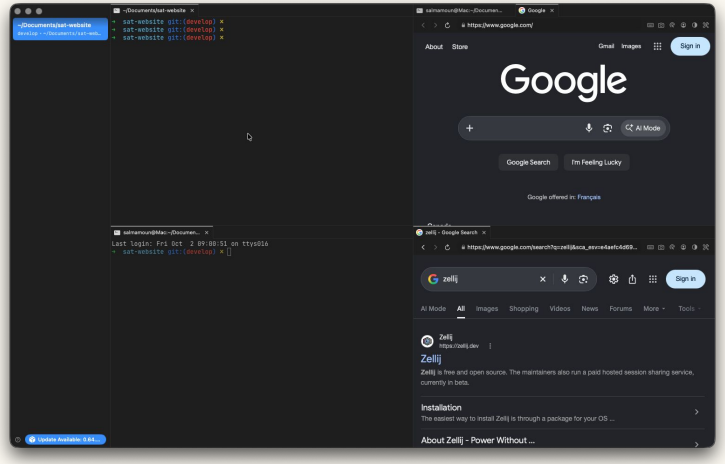
Inside VS Code

View › Terminal



Many at once

tmux, cmux, Zellij: one window, several terminals.



A **multiplexer** splits one window into panes, so nothing gets lost in a pile of windows.

Try: tmux, cmux, Zellij, or the split button in VS Code's terminal panel.

Why it's worth knowing if you use VS Code

Same folder as your code

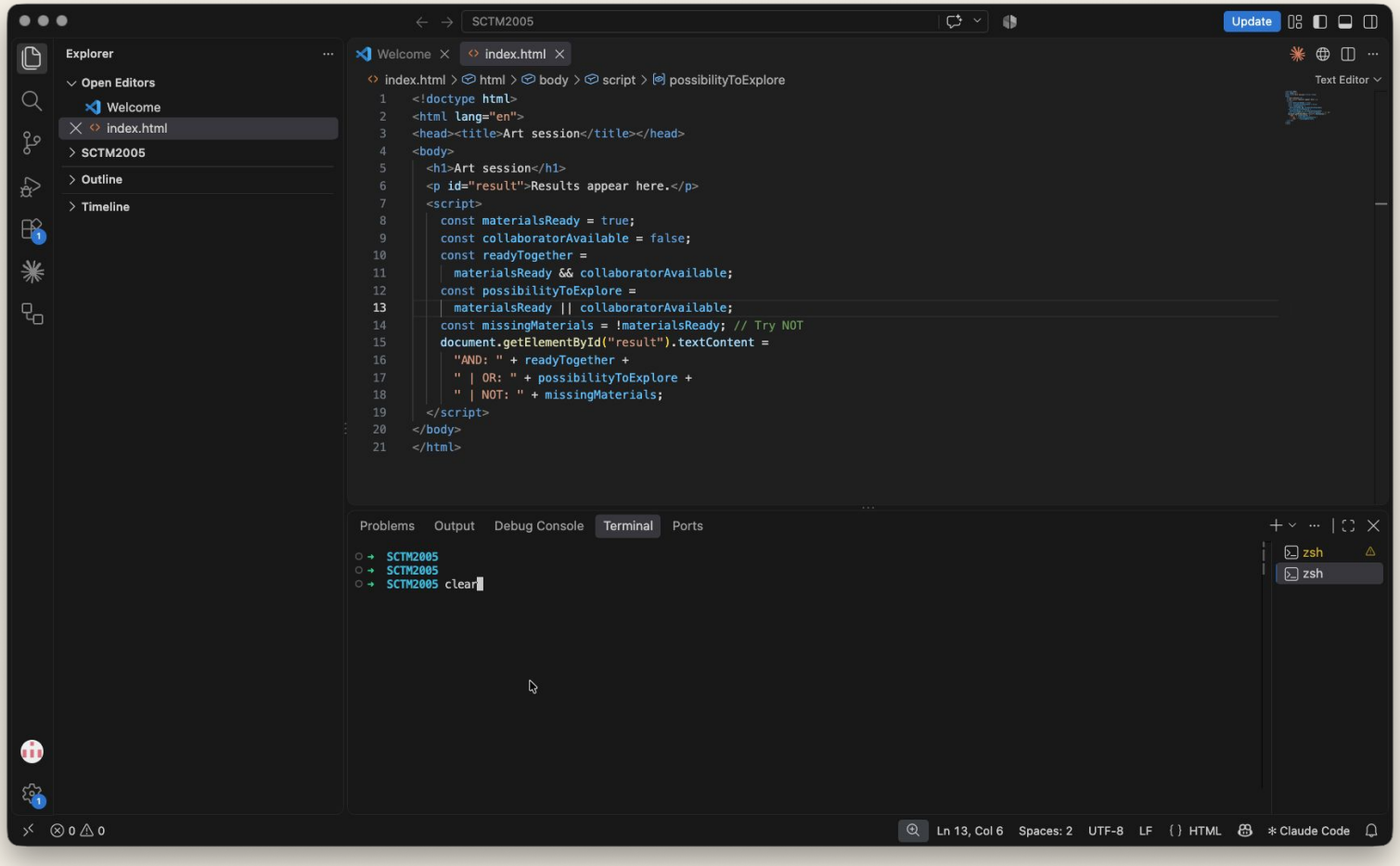
It opens right where your project lives.

Tools speak terminal

AI agents, npm, git and more all run here.

Errors show up here

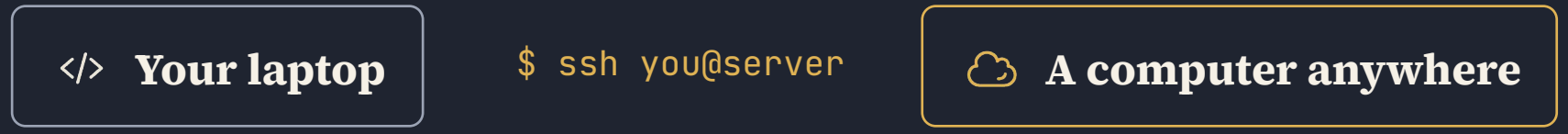
Reading the message is half the fix.



Most computers on the planet have no screen.

Servers, the cloud, a machine in a gallery installation.

People reach them through a terminal, from anywhere, with ssh.



Every command is a short sentence

```
$ ls -l Documents
```



The verb

ls: what to do (list files)

The how

-l: an option (a long, detailed list)

The what

Documents: what to do it to

Spaces separate the parts. Capitals matter: LS is not ls.

explainshell.com: paste a command, see each part explained

Moves to get you started

Finding your way

`pwd`	Where am I?
`ls`	What's in here?
`cd name`	Go into a folder
`cd ..`	Go back up one

Making things

`mkdir`	Make a folder
`touch`	Make an empty file
`code .`	Open it in VS Code

> ⚠ **rm** deletes a file or directory for good. There is no trash can, **read** before you press Enter.

Question break



Ask anything! Now's your time: make the most of your tuition feeeeeees.

Why does Windows look different?

Can I break my computer?

Shortcuts before the hands-on part

MOVE FASTER

`Tab`
Finish a file or folder name

`↑`
Bring back your last command

`Ctrl + ``
Show or hide VS Code's terminal

`code .`
Open this folder in VS Code

STOP SOMETHING OR GET UNSTUCK

`Ctrl + C`
Stop whatever is running

`q`
Quit a scroll view (git log, man)

`exit`
Close this shell session

`clear`
Wipe the screen clean

1 Folder | 2 Files | 3 Logic | 4 3D

Step 1: make a home for your project

zsh

```
$ cd Desktop          # pick a place
$ mkdir SCTM          # make a folder
$ cd SCTM             # step inside
$ code .              # open me in VS Code
```

1 Folder | 2 Files | 3 Logic | 4 3D

Step 2: give it three files

zsh · SCTM

```
$ touch index.html style.css script.js
$ ls
index.html script.js style.css
```

Windows PowerShell · SCTM

```
$ ni index.html, style.css, script.js
$ ls
index.html script.js style.css
```

HTML
What's on the page

CSS
How it looks

JavaScript
What it does

1 Folder | 2 Files | 3 Logic | 4 3D

Step 3: Open in code editor and add a little logic

index.html

```
<button>Click me</button>
<script src="script.js"></script>
```

script.js

```
let clicks = 0;
const btn = document.querySelector("button");
btn.addEventListener("click", () => {
  clicks = clicks + 1;
  btn.textContent = "Clicked " + clicks;
});
```

A variable
clicks remembers a number

An event
wait for a click

An update
add one, show the result

Open it in your browser:

```
open index.html
```

Windows: start index.html

Two tools you'll meet next: Node and git

Node.js

Runs JavaScript outside the browser, right in your terminal.

It comes with **npm**, a huge library of ready-made code, and **npx**, which runs a tool without installing it.

```
$ node -v
# v24.x? You have it
```

Get it: nodejs.org

git

Save points for a project folder, so you can go back when something breaks.

It remembers every version you save. **GitHub** is a website that stores those save points online.

```
$ git --version
# a version number? You have it
```

Get it: Mac offers to install it the first time · Windows: git-scm.com

1 Folder | 2 Files | 3 Logic | 4 3D

Step 4: level up to 3D with three.js

zsh

```
$ mkdir cube
$ cd cube && touch index.html
$ code index.html
# paste the example, save
$ npx serve
# open the link it prints
```

find some three.js code from the internet ! or ask an LLM to generate it.

copy source code from examples https://threejs.org/manual/#creating-a-scene

Why npx serve? Browsers won't run this kind of code from a file you double-click. It starts a tiny web server; **Ctrl + C** stops it. otherwise execute with **python3 -m http.server**

What you'll likely do with AI (LLM or agent) in the terminal

Agents live here

Claude Code, Codex CLI and Gemini CLI run in the terminal, inside your project folder.

Run what it suggests

AI answers often say npm install ... or npm run dev. Paste one line at a time, after reading it.

Paste errors back

Copy the whole red message into the chat. It's the best clue you have.

Save checkpoints with git

Commit before big AI changes, so you can always go back to a version that worked.

Why use git? (version control)



Sneak peek at the next tutorial: GitHub

git push sends your save points online:
a backup and a portfolio.

Many apps have a terminal side

`ffmpeg` video → GIF

```
ffmpeg -i clip.mov clip.gif
```

`ffmpeg` pull the audio out of a video

```
ffmpeg -i talk.mp4 talk.mp3
```

`magick` resize or convert images

```
magick photo.png -resize 50% photo.jpg
```

`pandoc` Markdown → Word or PDF

```
pandoc notes.md -o notes.docx
```

`open` any app or folder (Mac)

```
open -a "Google Chrome" index.html
```

`curl` download a file from the web

```
curl -O https://example.com/data.csv
```

Getting them: brew install ffmpeg on a Mac (Homebrew), winget on Windows.

I launch a script on my machine
A script is a recipe of commands

```
start.sh
#!/bin/bash
# open and run my project
cd ~/Desktop/SCTM
code .
open index.html
```

Type the same commands every day? Write them in a file once, then run the file.

> **script.js** tells your browser what to do.
> **start.sh** tells your computer what to do.

```
$ chmod +x start.sh      # once: allow it to run
$ ./start.sh             # run the whole recipe, top to bottom
```

It's OK not to know every detail. But know roughly what a line does,
because one short command can wipe your files with NO UNDO:

DO NOT RUN THIS

```
$ rm -rf ~/
```

rm remove -r and everything inside -f without asking ~/ your whole home folder

Do this instead

`$ ls notes.txt`	one file, checked first
`$ rm notes.txt`	Is it really the file you mean?
`$ mv notes.txt ~/.Trash/`	Normal remove: just that one file (no Trash, so check first)
	Or, on a Mac: send it to the Trash, so you can undo

When in doubt, paste it into **explainshell.com**, or ask an LLM to explain it first.

FURTHER READING

explainshell.com: paste a command, see each part explained

Oh My Zsh: themes and plugins for a friendlier terminal

tmux · **cmux** · **Zellij**: many terminals in one window

The Missing Semester of Your CS Education, MIT

> SHOUTOUT
>
> Julia Evans, *The Secret Rules of the Terminal* (wizardzines.com)

Questions? There are no dumb ones.

What would you use the terminal for now?